

변수(Variable)

- 변수 : 데이터를 저장하는 공간
- 할당 : = 의 오른쪽 값을 왼쪽 변수에 담는 것
 - 수학에서 사용하는 등호와 다른 의미
- 이미 할당된 변수에 다른 값을 넣을 수 있음

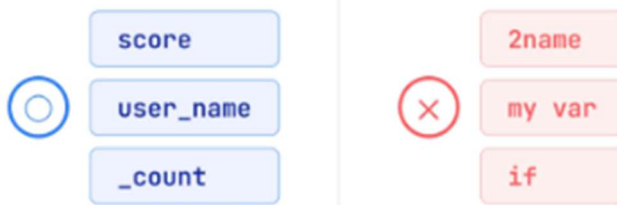


변수 이름 규칙

SS

- 영문 또는 _로 시작하며 숫자로는 시작 불가
- 공백은 쓸 수 없고 단어 사이는 _로 연결 (예: user_name)
- 한글도 되지만 영어를 권장
- 예약어(키워드)는 변수명으로 쓸 수 없음 (예: if, for, True)

※ 예약어(Keyword): if, for, True처럼 파이썬에서 문법적 용도로 정해 두어 변수 이름으로 사용할 수 없는 단어



기본 자료형

SS

자료형	의미	예시
int	정수	10, -3, 0
float	실수(소수점)	3.14, -0.5
str	문자열(글자)	"안녕", "AI"
bool	참/거짓	True, False

- 자료형은 데이터가 어떤 모양인지 알려주는 정보
- 같은 5라도 5(int)와 "5"(str)는 다르니 주의
- Python의 정수(int)는 크기 제한이 없음

int 90

float 3.14

str 'hello'

bool True

type()

- type()으로 변수의 자료형을 확인 가능

```
type(10)      # <class 'int'>
type(3.14)    # <class 'float'>
type("AI")    # <class 'str'>
type(True)    # <class 'bool'>
```

```
type("5")
```

```
>>> <class 'str'>
```

Python 사칙연산

```
# 변수에 값 담기
age = 27      # int
height = 175.5 # float
name = "김박피" # str
is_student = True # bool

# 사칙연산
print(age + 3)    # 30
print(age * 2)    # 54
print(age / 2)    # 13.5 (나누기 결과는 float)
print(age // 2)   # 13 (몫)
print(age % 2)    # 1 (나머지)
print(age ** 2)   # 729 (제곱)
```

age ← 27 int
height ← 175.5 float
name ← 김박피 str
is ← True bool

27
+3 → 30 *2 → 54 /2 → 13.5
// → 13 % → 1 ** → 729
+ 나누기는 항상 실수(float)

형 변환(Type Casting)

```
# 문자열 → 숫자 (가능한 경우)
print(int("27"))    # 27 (int)
print(float("3.14")) # 3.14 (float)

# 숫자 → 문자열
print(str(27))       # "27" (str)
print(str(3.14))     # "3.14" (str)

# 숫자 → 숫자 (타입 변환)
print(int(3.14))     # 3 (소수점 버림)
print(float(27))     # 27.0

# 숫자와 문자열 합치기
print("김박피" + str(27) + "살") # 김박피27살
print(f"김박피 {27}살")          # 김박피 27살
```

- 숫자와 문자열을 서로 합치고 싶다면 str()을 사용해 숫자를 문자열로 캐스팅 하거나 f-string을 사용할 수 있음

- int()로 실수를 정수로 캐스팅 할 때에는 소수점 아래를 버림

* f-string: 문자열 앞에 f를 붙이고 중괄호 {} 안에 변수나 표현식을 넣는 문자열 표기법

리스트(list)

- 리스트(list) : 여러 값을 순서대로 담는 자료구조
- 대괄호 `[]`를 이용해 만들 수 있으며, 값은 실패를 이용해 구분
- 인덱스(index) : 각 칸에 접근하기 위한 번호로 0부터 시작

```
scores = [90, 85, 70, 60]
fruits = ["사과", "배", "감"]
```



딕셔너리(dict)

- 딕셔너리(dict) : 키(key)와 값(value)을 쌍으로 담는 자료구조
- 중괄호 `{}`를 이용해 만들 수 있으며, `key: value` 형태로 작성함
- 키(key) : 값에 접근하기 위한 고유한 이름으로 중복될 수 없음

```
person = {"name": "김싸피", "age": 27, "job": "학생"}
```



list와 dict의 차이

구분	list	dict
모양	[v, v, v]	{k: v, k: v}
찾는 법	순서(인덱스)	이름(key)
언제	순서가 있는 데이터	이름표가 붙은 데이터
예	점수 목록	한 사람 정보

list - 번호로 찾기



dict - 이름으로 찾기



list 인덱싱

```
scores = [90, 85, 70, 60, 50]
#      0   1   2   3   4   (앞에서부터)
#      -5  -4  -3  -2  -1   (뒤에서부터)

print(scores[0]) # 90 (첫 번째)
print(scores[2]) # 70 (중간)
print(scores[-1]) # 50 (마지막)
print(scores[-3]) # 70 (뒤에서 3번째)
```

- 인덱싱(indexing) : []를 이용해 리스트의 특정 값에 접근하는 방법
- 양수 인덱스는 앞에서부터, 음수 인덱스는 뒤에서부터 셈
- [-1]을 이용하면 리스트의 길이를 몰라도 마지막 값에 접근할 수 있음

list 슬라이싱

```
scores = [90, 85, 70, 60, 50]
# [시작:끝]에서 끝 인덱스는 포함하지 않음
print(scores[0:3]) # [90, 85, 70], 인덱스 0부터 2까지
print(scores[:3]) # [90, 85, 70], 처음부터 인덱스 2까지
print(scores[2:]) # [70, 60, 50], 인덱스 2부터 마지막까지
print(scores[::2]) # [90, 70, 50], 두 칸 간격으로 선택
```

scores[0:3] : 인덱스 0부터 2까지의 값에 접근하며, 끝 인덱스인 3은 포함하지 않음



list 메서드

* 메서드(Method): 리스트나 딕셔너리 같은 객체에 점(.)을 붙여 호출하는 함수

```
scores = [90, 85, 70]
scores.append(60) # [90, 85, 70, 60], 마지막에 60 추가
scores.insert(1, 95) # [90, 95, 85, 70, 60], 인덱스 1에 95 삽입
scores.pop() # [90, 95, 85, 70], 마지막 값을 제거하고 60 반환
scores.pop(1) # [90, 85, 70], 인덱스 1의 값을 제거하고 95 반환
scores.remove(85) # [90, 70], 값이 85인 항목 제거
scores = [70, 90, 60, 85]
scores.sort() # [60, 70, 85, 90], 기존 리스트를 직접 정렬
print(sorted(scores)) # [60, 70, 85, 90], 정렬된 새로운 리스트 반환
```

- pop() : 값을 제거한 후 제거한 값을 반환
- remove() : 입력한 값을 찾아 제거
- sort() : 기존 리스트를 직접 정렬
- sorted() : 정렬된 새로운 리스트를 반환

dict 메서드

```
person = {"name": "김싸피", "age": 27, "job": "학생"}
# 값에 접근
print(person["name"])          # "김싸피", 키가 없으면 KeyError 발생
print(person.get("name"))      # "김싸피", 키가 없으면 None 반환
print(person.get("hobby", "없음")) # "없음", 키가 없으면 지정한 기본값 반환
# 키 존재 여부 확인
if "age" in person:            # True
    print(person["age"])        # 27
# 값 추가, 수정, 삭제
person["hobby"] = "독서"       # 키가 없으면 추가하고, 있으면 기존 값 수정
print(person.pop("job"))        # "학생", 키와 값을 삭제한 후 값 반환
```

- [] : 키가 없으면 KeyError 발생
- get() : 키가 없으면 None 또는 지정한 기본값을 반환
- in : 키의 존재 여부를 확인

list와 dict의 중첩 구조

```
# 딕셔너리 안에 리스트를 넣어 한 학생의 여러 점수 저장
student = {"name": "김싸피", "scores": [90, 85, 70]}
print(student["scores"][1]) # 85, scores에 저장된 리스트의 인덱스 1

# 리스트 안에 딕셔너리를 넣어 여러 학생의 정보 저장
students = [
    {"name": "김싸피", "age": 27},
    {"name": "민지", "age": 24},
]
print(students[0]["name"]) # 김싸피, 인덱스 0에 있는 학생의 name 값
print(students[1]["age"]) # 24, 인덱스 1에 있는 학생의 age 값
```

- 딕셔너리와 리스트는 서로 중첩 가능
- 학생 명단이나 API 응답처럼 여러 정보가 연결된 데이터를 표현할 때 사용

에러를 대하는 법

- 에러 메시지는 어느 줄에서 무엇이 잘못됐는지를 알려줌
- 에러를 잘 파악하는 개발자가 좋은 개발자

```
Traceback (most recent call last):
  File "main.py", line 3, in <module>
    print(name + age)
TypeError: can only concatenate str
```


마지막 줄과
line 번호부터 읽기

Java와 Python 비교 - 변수와 자료형

* 컴파일(Compile): 소스 코드를 실행 가능한 형태로 미리 변환하고 문법이나 자료형 오류를 검사하는 과정

구분	Java	Python
변수 선언	<code>int n = 10;</code> 자료형을 직접 명시	<code>n = 10</code> 자료형을 자동으로 결정
타입 시스템	정적 타입, 컴파일 시 자료형 결정	동적 타입, 실행 시 자료형 결정
문자열	<code>String s = "hi";</code>	<code>s = "hi"</code>
리스트, 배열	<code>int[] a = {1, 2, 3};</code>	<code>a = [1, 2, 3]</code>
맵, 딕셔너리	<code>Map<String, Integer> m = new HashMap<>();</code>	<code>m = {"a": 1}</code>
문장 종결	세미콜론 사용	줄바꿈으로 구분하며 세미콜론 생략 가능
코드 블록	중괄호 {}를 이용해 구분	들여쓰기를 이용해 구분

조건문

- 조건문: 조건의 참, 거짓에 따라 실행할 코드를 결정
- 조건문에 포함되는 코드는 공백 4칸으로 들여쓰기

```
if 조건:
    조건이 참이면 실행
elif 다른 조건:
    앞의 조건이 거짓이고 다른 조건이 참이면 실행
else:
    모든 조건이 거짓이면 실행
```



비교 연산자와 논리 연산자

SSAFY

비교 연산자 - True / False

연산자	의미	예시	결과
<code>==</code>	같음	<code>5 == "5"</code>	False (종류가 다른 값)
<code>!=</code>	다름	<code>5 != "5"</code>	True
<code>></code>	초과	<code>10 > 5</code>	True
<code><</code>	미달	<code>10 < 5</code>	False
<code>>=</code>	예상	<code>10 >= 10</code>	True
<code><=</code>	미하	<code>5 <= 10</code>	True

```
age = 27
if age >= 20:
    print("성인입니다") # 출력됨
if age == 27:
    print("정확히 27살입니다")
```

주의! 숫자와 문자열처럼 종류가 다른 값은 반드시 비교하지 않음 → `5 == "5"`는 False

거짓 값(Falsey)

```
# 0, 빈 문자열, 빈 리스트, None은 거짓으로 평가
if 0:
    print("실행되지 않음")

if "":
    print("실행되지 않음")

scores = []
if scores:
    print("점수 있음") # 빈 리스트이므로 거짓
else:
    print("점수 없음") # 실행

name = "김차픽"
if name:
    print("이름 있음") # 비어 있지 않은 문자열이므로 참
```

- 0, "", [], None : 조건식에서 거짓으로 평가
- if scores : 리스트에 값이 있는지 확인
- if not scores : 리스트가 비어 있는지 확인
- len(scores) == 0 대신 if not scores를 이용해 빈 리스트인지 확인 가능

반복문

- for : 반복할 대상의 값을 변수에 하나씩 대입하며 반복
- range(n) : 0부터 n-1까지의 숫자를 생성

```
for 변수 in 반복할 대상:
    반복할 코드

for i in range(3): # i에 0, 1, 2를 순서대로 대입
    반복할 코드
```



range() 함수

```
for i in range(5):
    print(i) # i = 0, 1, 2, 3, 4

for i in range(1, 5):
    print(i) # i = 1, 2, 3, 4

for i in range(0, 10, 2):
    print(i) # i = 0, 2, 4, 6, 8

for i in range(10, 0, -1):
    print(i) # i = 10, 9, ..., 1 (역순)
```

코드	결과	설명
range(5)	0, 1, 2, 3, 4	0부터 4까지
range(1, 5)	1, 2, 3, 4	1부터 4까지
range(0, 10, 2)	0, 2, 4, 6, 8	2칸씩 증가
range(10, 0, -1)	10, 9, ..., 1	내림차순

break와 continue

```
# break를 이용해 반복문 종료
for i in range(10):
    if i == 3:
        break # i가 3이면 반복문 종료
    print(i)
# 출력: 0, 1, 2

# continue를 이용해 현재 반복 건너뛰기
for i in range(5):
    if i == 2:
        continue # i가 2이면 다음 반복으로 이동
    print(i)
# 출력: 0, 1, 3, 4

# break 없이 반복이 끝나면 else 실행
for i in range(3):
    print(i)
else:
    print("반복 완료") # break가 실행되지 않았으므로 출력
```

- **break**: 반복문을 즉시 종료
- **continue**: 현재 반복을 중단하고 다음 반복으로 이동
- **for-else**: break가 실행되지 않고 반복이 끝나면 else 블록을 실행

함수

- **함수(function)**: 특정 작업을 수행하는 코드를 하나로 묶은 것
- **def**: 함수를 정의
- **return**: 함수의 실행 결과를 반환
- 정의한 함수는 이름을 이용해 여러 번 호출 가능

```
def 함수_이름(입력값):
    처리할 코드
    return 결과

함수_이름(값) # 함수 호출
```



함수의 호출 스택과 스코프

* 호출 스택(Call Stack): 호출된 함수의 실행 정보가 순서대로 쌓이고, 함수가 종료되면 최근에 호출된 함수부터 제거되는 구조

```
def greet(name):
    message = f"안녕, {name}!" # 지역변수 message
    print(message)
    return message

result = greet("김싸피") # 안녕, 김싸피!
print(result) # 안녕, 김싸피!
```

1. **greet("김싸피")**를 호출해 함수 실행
2. 함수 안에서 지역 변수 **message** 생성
3. **return message**로 값을 반환하고 함수 종료
4. 반환된 값을 **result**에 저장
5. 함수가 종료되면 지역 변수 **message**에 접근할 수 없음


```
global_var = 100 # 전역 변수, 함수 안과 밖에서 접근 가능

def my_func():
    local_var = 50 # 지역 변수, 함수 안에서만 접근 가능
    print(global_var) # 100
    print(local_var) # 50

my_func()
print(global_var) # 100
print(local_var) # NameError, 함수 밖에서 접근할 수 없음
```

변수 종류	정의 위치	접근 범위	생명 주기
전역 변수	함수 밖	함수 안과 밖	프로그램이 종료될 때까지 유지
지역 변수	함수 안	변수를 정의한 함수 안	함수가 종료될 때까지 유지

함수 안에서 정의한 지역 변수는 함수가 종료되면 함수 밖에서 접근할 수 없음

if문 예제

```
score = 75

if score >= 90:
    print("A")
elif score >= 80:
    print("합격")
else:
    print("불합격")
# 출력: 합격
```

- **>=** : 왼쪽 값이 오른쪽 값보다 크거나 같은지 비교
- 조건은 위에서부터 확인하며, 처음으로 참인 조건의 분기 하나만 실행
- 모든 조건이 거짓이면 **else** 분기를 실행

```
score = 95
if score >= 90:
    print("A")
elif score >= 80: print("합격")
else: print("불합격")
```

→ 합격
A

for문 예제

```
scores = [90, 85, 70, 60]

total = 0
for score in scores:
    total = total + score # score에 각 점수를 순서대로 대입
                        # 각 점수를 total에 더함

print(total) # 305, 합계
print(total / len(scores)) # 76.25, 평균
```

- 리스트의 길이와 관계없이 같은 **for** 문으로 합계를 계산할 수 있음

평균 함수 만들기

```
def average(numbers):
    return sum(numbers) / len(numbers)

print(average([90, 85, 70, 60])) # 76.25
print(average([100, 100]))       # 100.0
```

- `average()` : 숫자의 합계를 개수로 나누어 평균을 계산
- `sum()` : 리스트에 있는 숫자의 합계를 계산
- `len()` : 리스트에 있는 값의 개수를 반환
- 정의한 함수는 여러 숫자 리스트에 재사용 가능

Java와 Python 비교 - 제어문과 함수

SSAFY

구분	Java	Python
조건문	if (x > 0) { ... }	if x > 0: (콜론+들여쓰기)
else if	else if (...) { }	elif ...:
for 반복	for (int i=0; i<n; i++)	for i in range(n):
요소 순회	for (int x : arr) { }	for x in arr:
함수 정의	int add(int a, int b){ return a+b; }	def add(a, b): return a + b
반환 타입	명시 필요 (int, void)	불필요 (자동 추론)
불러옴	boolean, true / false	bool, True / False

API

API의 동작 방식은 식당의 주문 과정에 비유할 수 있음

- 손님 : 클라이언트(client)
- 메뉴판 : API 문서, 요청할 수 있는 기능과 사용 방법을 안내
- 주문 : 요청(request)
- 주방 : 서버(server)
- 음식 : 응답(response)
- 손님이 음식의 조리 과정을 몰라도 주문할 수 있듯, 클라이언트는 서버 내부 처리 과정을 몰라도 API를 통해 요청을 보내고 응답을 받을 수 있음



Request와 Response

Request / Response

- 클라이언트(client) : 서버에 요청(request)을 전송
- 서버(server) : 요청을 처리한 후 응답(response)을 반환
- URL : 요청할 자원의 위치를 나타내는 주소



HTTP 메서드

- HTTP 요청은 목적에 따라 서로 다른 메서드를 사용함
 - GET : 데이터를 조회
 - POST : 데이터를 전송하거나 생성을 요청
 - PUT : 기존 데이터 전체를 수정
 - PATCH : 기존 데이터 일부를 수정
 - DELETE : 데이터를 삭제
 - OPTIONS : 사용할 수 있는 메서드를 확인

메서드	식당 비유	용도
GET	"메뉴판을 보여 주세요"	조회
POST	"이 주문을 접수해 주세요"	전송, 생성

상태 코드

- 상태 코드 : 서버가 요청을 처리한 결과를 숫자로 나타낸 값
- 식당에서 주문 처리 결과를 알려주는 영수증에 비유할 수 있음

코드	의미	식당 비유
200	요청 성공	음식이 정상적으로 제공됨
404	요청한 자원을 찾을 수 없음	주문한 메뉴가 없음
500	서버 내부 오류	주방에서 문제가 발생함

- 성공을 나타내는 상태 코드이면 응답 데이터를 처리
- 오류를 나타내는 상태 코드이면 코드에 따라 원인을 확인

201이면 created의 의미로 post일때

400: bad request, 401: unauthorized, 403: forbidden, 405: method error

500: internal server error

requests 라이브러리

- **requests**: 파이썬에서 HTTP 요청을 보내고 응답을 받을 때 사용하는 외부 라이브러리
- 설치: `pip install requests`
- **requests.get(URL)**: 지정된 URL에 GET 요청을 보내고 응답 객체를 반환

```
import requests

response = requests.get("https://example.com")
```

* 라이브러리(Library): 미리 만들어 둔 기능을 다른 프로그램에서 불러와 사용할 수 있도록 모아 놓은 코드
외부 라이브러리: 파이썬에 기본으로 포함되지 않아 별도로 설치해야 하는 라이브러리로, requests가 대표적인 예
pip: 파이썬 라이브러리를 설치하고 관리하는 도구

response 객체

SSAFY

* 속성(Attribute): status_code나 text처럼 객체에 점(.)을 붙여 접근하는 값으로, 괄호를 붙여 호출하는 메서드와 구분됨

- **requests.get()**: 서버의 응답을 Response 객체로 반환

속성, 메서드	자료형	의미	예시
status_code	int	HTTP 상태 코드	200
text	str	문자열 형태의 응답 본문	['name': '김씨']
json()	해석된	JSON 응답을 파싱해 파이썬 객체로 반환	['name': '김씨']
headers	CaseInsensitiveDict	응답 헤더	{'Content-Type': 'application/json'}
elapsed	timedelta	응답에 걸린 시간	0:00:00.500000
url	str	실제로 요청한 URL	https://example.com/api

```
response = requests.get(
    "https://jsonplaceholder.typicode.com/users",
    timeout=5,
)

print(response.status_code)      # 200
print(type(response.text))       # <class 'str'>
print(type(response.json()))     # <class 'list'>
print(response.elapsed.total_seconds()) # 0.51, 초
```

- **response.text**: 응답 본문을 문자열로 확인
- **response.json()**: JSON 응답을 파싱해 리스트나 딕셔너리와 같은 파이썬 객체로 반환
- JSON 데이터를 코드에서 다룰 때에는 **response.json()**를 사용

Requests 테스트

- **JSONPlaceholder**: 회원가입이나 API 키 없이 사용할 수 있는 공개 연습용 API
- 요청 URL: <https://jsonplaceholder.typicode.com/users>

```
import requests

URL = "https://jsonplaceholder.typicode.com/users"

# GET 요청
response = requests.get(URL, timeout=5)

# 상태 코드 출력
print(response.status_code) # 200, 요청 성공
```

Requests 테스트

- `response.text`: JSON 응답 본문을 문자열로 반환
- `response.json()`: JSON 응답을 리스트나 딕셔너리와 같은 파이썬 객체로 변환
- 예제의 응답 데이터는 여러 사용자 딕셔너리가 담긴 리스트

```
if response.status_code == 200:
    data = response.json() # JSON 응답을 파이썬 객체로 변환

    print(type(data))      # <class 'list'>
    print(len(data))      # 10, 사용자 수
```

예외 처리

※ 예외(Exception): 프로그램 실행 중 문제가 발생했음을 알리는 신호로, 처리하지 않으면 정상적인 실행 흐름이 중단됨

- 네트워크 요청은 응답 지연, 연결 실패, HTTP 오류, JSON 변환 실패 등의 예외가 발생할 수 있음
- `except Exception`으로 모든 예외를 처리하기보다 원인별로 나누어 처리

```
import json
import requests

try:
    response = requests.get(URL, timeout=5)
    response.raise_for_status() # 400번대 또는 500번대이면 HTTPError 발생
    data = response.json()

except requests.exceptions.Timeout:
    print("5초 안에 응답을 받지 못함")
    data = []

except requests.exceptions.ConnectionError:
    print("서버에 연결할 수 없음")
    data = []

except requests.exceptions.HTTPError:
    print(
        f"HTTP 오류: {response.status_code} {response.reason}"
    )
    data = []

except json.JSONDecodeError:
    print("응답을 JSON으로 변환할 수 없음")
    data = []
```

JSON(JavaScript Object Notation)

S

JSON이란

- 프로그램 간에 데이터를 주고받거나 저장할 때 사용하는 텍스트 기반 데이터 형식
- 사람이 읽기 쉬우며 다양한 프로그래밍 언어에서 지원



JSON 구조

```
[
  {
    "id": 1,
    "name": "Leanne Graham",
    "email": "leanne@april.biz",
    "address": { "city": "Gwenborough", "zipcode": "92998" },
    "company": { "name": "Romaguera-Crona" }
  },
  { "id": 2, "name": "Ervin Howell", "email": "ervin@melissa.tv" }
]
```

- 배열(array) : 여러 사용자의 객체를 순서대로 저장
- 객체(object) : 사용자 한 명의 정보를 키와 값으로 저장
- 중첩 객체 : address, company처럼 객체 안에 저장된 객체
- response.json()으로 변환하면 배열은 리스트, 객체는 딕셔너리가 됨

값 접근하기

- 첫 번째 사람의 도시 이름을 꺼내려면?



JSON 데이터 접근

키로 한 칸씩 들어가기

- 리스트 : 인덱스를 이용해 값에 접근
- 딕셔너리 : 키를 이용해 값에 접근
- 중첩된 데이터 : 바깥쪽 구조부터 안쪽 구조까지 순서대로 접근

```
first_user = data[0] # 첫 번째 사용자의 딕셔너리
print(first_user["name"]) # 이름
print(first_user["email"]) # 이메일
print(first_user["address"]["city"]) # 주소에 저장된 도시
```

- data[0] : 리스트에서 첫 번째 사용자에 접근
- first_user["name"] : 딕셔너리에서 name 키의 값에 접근
- first_user["address"]["city"] : address 키에 접근한 후 city 키에 접근
- 딕셔너리는 .name이 아닌 ["name"] 형식으로 값에 접근

get() 메서드

- `dict["키"]`: 키가 없으면 `KeyError` 발생
- `dict.get("키")`: 키가 없으면 `None` 반환
- `dict.get("키", 기본값)`: 키가 없으면 지정한 기본값 반환

```
user = {
    "id": 1,
    "name": "김싸피",
    "email": "kim@example.com",
}

# user["phone"] # KeyError, phone 키가 없음
print(user.get("phone")) # None
print(user.get("phone", "N/A")) # N/A, 지정한 기본값
print(user.get("name", "?")) # 김싸피, name 키의 값
```

- API 응답에서 선택 항목을 확인할 때 `get()`을 사용하면 키가 없어도 기본값으로 처리 가능

필요한 값 추출

- API 응답에는 현재 작업에 필요하지 않은 필드도 포함될 수 있음
- 반복문으로 각 데이터에 접근해 필요한 키의 값만 추출

```
rows = []
for user in data:
    rows.append({
        "name": user["name"],
        "email": user["email"],
        "city": user["address"]["city"],
        "company": user["company"]["name"],
    })

print(rows[0])
# {'name': 'Leanne Graham', 'email': 'leanne@april.biz',
#  'city': 'Gwenborough', 'company': 'Romaguera-Crona'}
```

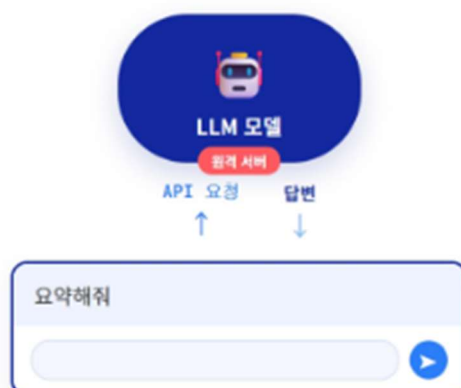
LLM 호출

SSAFY

* 대규모 언어 모델(LLM): 많은 양의 언어 데이터를 학습하고, 앞에 나온 내용을 바탕으로 다음 토큰을 예측해 문장을 생성하는 모델

챗봇은 어디서 답을 만들까

- ChatGPT, Claude와 같은 LLM은 외부 AI 서버에서 실행
- API를 통해 AI 서버에 요청을 보내고 응답을 받음
- `requests`: API 요청을 전송
- `JSON`: 요청과 응답 데이터를 표현



컨텍스트, RAG, 에이전트(Agent)

S

- **컨텍스트** : 모델이 알지 못하는 최신 정보나 내부 정보를 질문과 함께 전달
- **RAG** : 질문과 관련된 자료를 검색해 컨텍스트로 전달
- **에이전트(Agent)** : 필요한 도구를 사용해 작업을 수행



API 키 보안

* 환경 변수(Environment Variable): 운영체제나 실행 환경에 저장해 두고 프로그램에서 읽어 사용하는 값

- **API 키** : API 사용자를 식별하고 인증하는 비밀 정보
- API 키가 노출되면 무단 사용으로 요금이 청구될 수 있음
- API 키를 코드에 직접 작성하지 않음
- GitHub와 같은 공개 저장소에 API 키를 업로드하지 않음
- 환경 변수인 `os.environ`이나 별도의 비밀 파일로 관리



키 보관법 : 1. Os 환경변수 2. .env

실제 호출 형태 (의사코드)

SSAFY

* 의사코드(Pseudocode): 실제 실행보다 코드의 형식과 처리 흐름을 설명하려고 작성한 코드

- API 키를 이용해 API 서버에 POST 요청을 전송
- JSON 응답의 키에 순서대로 접근해 답변을 추출
- 아래 코드는 요청 형식만 보여 주는 의사코드

```
headers = {
    "Authorization": f"Bearer {os.environ['OPENAI_API_KEY']}"
}

body = {
    "model": "some-model",
    "messages": [
        {
            "role": "user",
            "content": prompt,
        },
    ],
}

response = requests.post(
    API_URL,
    headers=headers,
    json=body,
    timeout=30,
)

answer = response.json()[0]["choices"][0]["message"]["content"]
```


LLM API Request

SSAFY

* 토큰(Token): 모델이 텍스트를 읽고 생성할 때 사용하는 단위로, 단어나 글자의 일부가 하나의 토큰이 될 수 있음

- LLM API에 POST 요청의 요청 본문

```
body = {
  "model": "some-model",
  "messages": [
    {
      "role": "user",
      "content": "prompt",
    }
  ],
  "temperature": 0.7,
  "max_tokens": 500,
}
```

필드	자료형	의미	예시
model	str	사용할 모델	"some-model"
messages	list	역할과 내용으로 구성된 대화 목록	[{"role": "user", ...}]
temperature	float	응답의 무작위성을 조절	0.7
max_tokens	int	출력할 수 있는 최대 토큰 수	500

- messages**: 역할과 내용을 담은 항목을 순서대로 전달
- temperature**: 값이 낮을수록 응답이 비교적 일관되고, 높을수록 다양한 응답을 생성

LLM API Response

- 응답 본문은 JSON 형식이며, 키와 인덱스에 순서대로 접근해 필요한 값을 추출

```
{
  "model": "some-model",
  "usage": {"prompt_tokens": 50, "completion_tokens": 25, "total_tokens": 75},
  "choices": [
    {
      "index": 0, "message": {"role": "assistant", "content": "모델이 생성한 답변"}, "finish_reason": "stop"
    }
  ]
}

data = response.json()
choices = data["choices"]

if choices:
  answer = choices[0]["message"]["content"]
```

필드 경로	의미	자료형
choices[0]["message"]["content"]	응답 내용	str
usage["total_tokens"]	사용한 전체 토큰 수	int
choices[0]["finish_reason"]	응답 생성이 끝난 이유	str

- choices**: 생성된 응답을 담은 리스트
- choices[0]**: 첫 번째 응답
- choices**가 비어 있지 않은지 확인한 후 첫 번째 응답에 접근

핵심 키워드

주제	키워드	핵심
변수, 자료형	=, type(), int, str, bool	데이터 저장, 자료형 확인, 형 변환
자료구조	list, dict	인덱스 또는 키로 값에 접근
인덱싱	[0], [-1], [a:b]	인덱스는 0부터 시작하여 끝 인덱스는 포함하지 않음
조건, 반복, 함수	if, elif, for, def	조건 분기, 반복 실행, 함수 정의
API, HTTP	requests, status_code	요청과 응답, 상태 코드 확인
JSON 파싱	.json(), dict, list	JSON 응답을 파이썬 객체로 변환
안전한 접근	.get("키", 기본값)	키가 없으면 지정된 기본값을 반환

오늘 공부한 내용 요약 및 정리

- 숫자와 문자열을 연결할 때에는 `str()`이나 `f-string`을 사용
- 리스트는 0부터 시작하는 인덱스로 접근하고, 딕셔너리는 키로 접근
- 슬라이싱은 끝 인덱스를 포함하지 않음
- 코드 블록은 들여쓰기로 구분
- `append()` : 기존 리스트를 수정하고 `None`을 반환하므로 결과를 다시 저장하지 않음
- API : 프로그램 간에 요청과 응답으로 데이터를 주고받는 방식
- `requests.get()`으로 GET 요청을 보내고 `response.json()`으로 JSON 응답을 파싱
- 파싱한 JSON 데이터는 키와 인덱스로 접근
- `get("키", 기본값)` : 키가 없으면 지정한 기본값을 반환
- LLM 응답은 요청과 응답 구조를 사용하며, 응답에서 `content` 값을 추출
- API 키는 코드에 직접 작성하지 않고 환경 변수로 관리